



TRƯỜNG ĐẠI HỌC KINH TẾ KỸ THUẬT CÔNG NGHIỆP
Khoa Công Nghệ Thông Tin
----------

LẬP TRÌNH DI ĐỘNG

CHƯƠNG 5: INTENT VÀ SERVICE





Chương 5 – Intent và Service



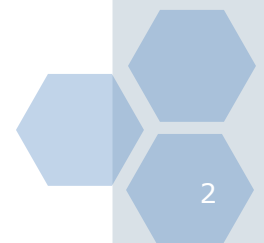
TỔNG QUAN INTENT



SỬ DỤNG INTENT TRAO ĐỔI DỮ LIỆU



BROADCAST RECEIVERS





UNESI

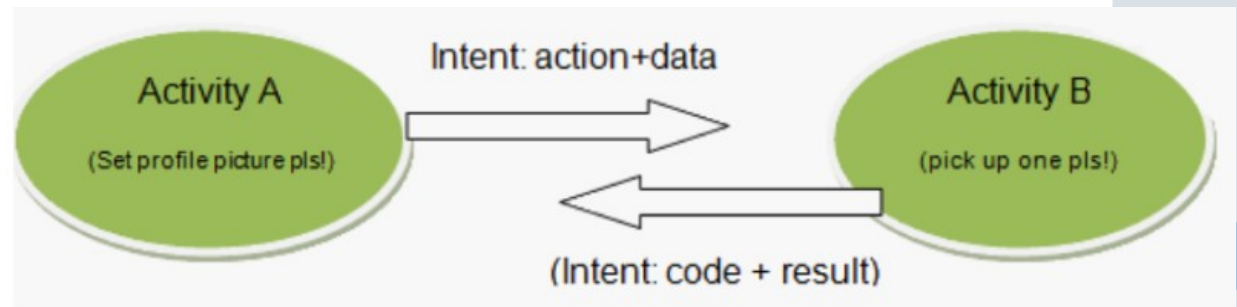


1.1 TỔNG QUAN INTENT



1.1.1 INTENT & INTENT SERVICE

- **Intent** là chuẩn giao tiếp giữa các thành phần trong Android OS (activity, service, provider, receiver)
- **Intent giống một lá thư**: các thông tin cần thiết được đóng gói bên trong một intent (địa chỉ + nội dung)
- **Intent service** là dịch vụ hệ thống, vai trò như người đưa thư: chuyển intent tới thành phần nhận phù hợp nhất (chiếu theo địa chỉ ghi trong intent)





1.1.1 INTENT & INTENT SERVICE

- Ví dụ sử dụng Intent**

```
Intent x = new Intent(this, Login.class);  
x.putExtra("loginname", "abcxyz");  
startActivity(x);
```

**mở activity
Login với dữ
liệu gửi kèm
loginname là
abcxyz**

```
Intent i = new Intent(Intent.ACTION_VIEW);  
i.setData(Uri.parse("http://uneti.edu.vn"));  
startActivity(i);
```

mở trang web uneti.edu.vn

**mở activity quay số với
số 0912102165 điền sẵn**

```
Intent dialIntent = new  
    Intent(Intent.ACTION_DIAL, Uri.parse("tel:0912102165"));  
startActivity(dialIntent);
```



1.1.1 INTENT & INTENT SERVICE

- Dùng Intent và nhận dữ liệu trả về

```
private static int TAKE_PICTURE = 1;
private Uri outputFileUri;
private void TakePhoto() {
    Intent intent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
    File file = new File(Environment.getExternalStorageDirectory(), "test.jpg");

    outputFileUri = Uri.fromFile(file);
    intent.putExtra(MediaStore.EXTRA_OUTPUT, outputFileUri);
    startActivityForResult(intent, TAKE_PICTURE);
}
```

bật ứng dụng camera chụp ảnh và trả về ảnh chụp

```
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data){

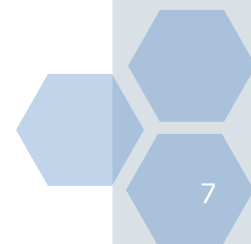
    if (requestCode == TAKE_PICTURE){
        ShowMessage(outputFileUri.toString());
    }

}
```



1.1.2 INTENT FILTER (Bộ lọc Intent)

- Activity, Service và Broadcast receiver sử dụng intent filter để thông báo cho hệ thống biết các dạng intent mà nó có thể xử lý
- Phân giải intent (intent resolution): khi nhận được một intent, hệ thống tiến hành chọn activity phù hợp nhất với intent đó theo ưu tiên sau:
 - Action trong intent
 - Chuỗi tham số (URI trong phần data)
 - Category của intent



1.1.2 INTENT FILTER (Bộ lọc Intent)

```
<activity android:name=".Dialer"
          android:label="@string/app_name">
  <intent-filter android:priority="100" >
    <action android:name="android.intent.action.DIAL" />
    <category android:name="android.intent.category.DEFAULT" />
    <data android:scheme="tel"/>
  </intent-filter>
</activity>
```

- Đoạn mã XML trên đăng kí với hệ thống một activity “lắng nghe” việc người dùng muốn thực hiện cuộc gọi bằng cách chỉ định thuộc tính action trong intent-filter

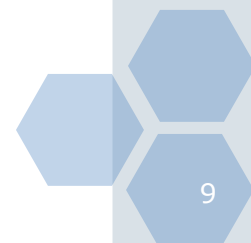




1.1.3 PHÂN LOẠI INTENT

Intent tường minh

- Sử dụng thành phần component để chỉ định rõ đối tượng sẽ thực thi
- Sử dụng các phương thức:
 - `setComponent(ComponentName)`
 - `setClass(Context, Class)`
 - `setClassName(Context, String)`
 - `setClassName(string, string)`
- Chỉ được dùng để gọi các activity trong cùng một ứng dụng
- Dữ liệu trao đổi nên chuyển vào phần extras





1.1.3 PHÂN LOẠI INTENT

Intent tường minh

```
Intent intent = new Intent();  
  
intent.setClassName("ten_package", "ten_class");  
  
// chuẩn bị dữ liệu trước khi gửi  
  
...  
  
startActivity(intent);
```

Hoặc

```
Intent i = new Intent(this, Activity2.class);  
  
// chuẩn bị dữ liệu trước khi gửi  
  
...  
  
startActivity(intent);
```



1.1.3 PHÂN LOẠI INTENT

Intent không tường minh

- Dùng các thành phần action, category,...
- Hệ thống tự động xác định đối tượng phù hợp nhất để đáp ứng với Intent đó (theo nguyên tắc “phân giải intent” đã trình bày ở slide 8)
- Dùng để giao tiếp với các dịch vụ hệ thống hoặc dịch vụ do bên thứ ba cung cấp:
 - Gọi activity: `startActivity` / `startActivityForResult`
 - Gọi service: `startService` / `bindService`
 - Gửi broadcast: `sendBroadcast` / `sendOrderedBroadcast` / `setStickyBroadcast`

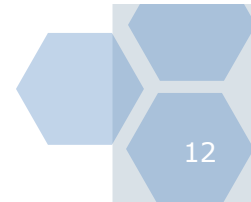




1.1.3 PHÂN LOẠI INTENT

Một số trường hợp sử dụng Intent không tường minh

Định dạng	Action	Mô tả
tel:phone_number	ACTION_VIEW	Mở Dial form (chưa gọi)
tel:phone_number	ACTION_CALL	Thực hiện gọi tới số phone
http://web_address https://web_address	ACTION_VIEW	Mở trình duyệt web với địa chỉ được cấp
"some_words" (string) http://web_address https://web_address	ACTION_WEB_SEARCH	Thực hiện search
sms://	ACTION_SENDTO	Gửi tin nhắn
geo:latitude,longitude geo:latitude,longitude?z=zoom geo:0,0?q=my+street+address geo:0,0?q=business+near+city	ACTION_VIEW	Mở ứng dụng Maps và chỉ tới vị trí được xác định





1.1.3 PHÂN LOẠI INTENT

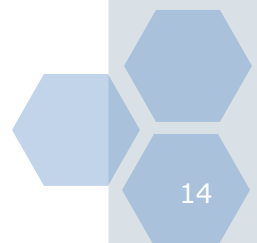
Các Action được định nghĩa sẵn

Built-in Standard Actions	
<u>ACTION_MAIN</u> <u>ACTION_VIEW</u> <u>ACTION_ATTACH_DATA</u> <u>ACTION_EDIT</u> <u>ACTION_PICK</u> <u>ACTION_CHOOSER</u> <u>ACTION_GET_CONTENT</u> <u>ACTION_DIAL</u> <u>ACTION_CALL</u> <u>ACTION_SEND</u>	<u>ACTION_ANSWER</u> <u>ACTION_INSERT</u> <u>ACTION_DELETE</u> <u>ACTION_RUN</u> <u>ACTION_SYNC</u> <u>ACTION_PICK_ACTIVITY</u> <u>ACTION_SEARCH</u> <u>ACTION_WEB_SEARCH</u> <u>ACTION_FACTORY_TEST</u> <u>ACTION_SENDTO</u>
Built-in Standard Broadcast Actions	
<u>ACTION_TIME_TICK</u> <u>ACTION_TIME_CHANGED</u> <u>ACTION_TIMEZONE_CHANGED</u> <u>ACTION_BOOT_COMPLETED</u> <u>ACTION_PACKAGE_ADDED</u> <u>ACTION_PACKAGE_CHANGED</u> <u>ACTION_PACKAGE_REMOVED</u>	<u>ACTION_PACKAGE_RESTARTED</u> <u>ACTION_PACKAGE_DATA_CLEARED</u> <u>ACTION_UID_REMOVED</u> <u>ACTION_BATTERY_CHANGED</u> <u>ACTION_POWER_CONNECTED</u> <u>ACTION_POWER_DISCONNECTED</u> <u>ACTION_SHUTDOWN</u>



1.1.4 CÁC THÀNH PHẦN CỦA INTENT

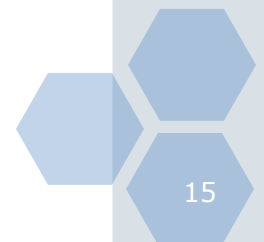
Thuộc tính chính	Thuộc tính phụ
action -tên (string) của action mà Intent sẽ yêu cầu thực hiện -có thể là action được Android định nghĩa sẵn (built-in standard action) hoặc do người lập trình tự định nghĩa	category -thông tin về nhóm của action type -định dạng kiểu dữ liệu (chuẩn MIME) -thường được tự động xác định
data -dữ liệu mà Activity được gọi sẽ xử lý -định dạng Uri (thông qua hàm Uri.parse(data))	component -chỉ định cụ thể lớp sẽ thực thi Activity -khi được xác định, các thuộc tính khác trở thành không bắt buộc (optional)
	extras -chứa tất cả các cặp (key,value) do ứng dụng thêm vào để truyền qua Intent (cấu trúc Bundle)
http://developer.android.com/reference/android/content/Intent.html	





1.1.4 CÁC THÀNH PHẦN CỦA INTENT

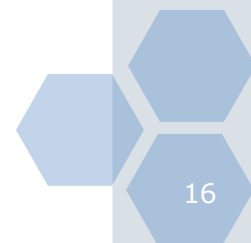
- **Component name:** tên class xử lý intent (ví dụ: “com.example.project.app.MyActivity1”)
- **Action:** tên các hành động mà intent yêu cầu thực hiện (ví dụ: action_view, action_call,...)
- **Data:** dữ liệu yêu cầu được xử lý, dữ liệu này thường được biểu diễn dưới dạng URI (ví dụ: "tel:216-555-1234", "http://uneti.edu.vn",...)
 - Trường hợp dữ liệu phức tạp hoặc không cố định, người ta thường đẩy vào phần extras





1.1.4 CÁC THÀNH PHẦN CỦA INTENT

- **Type**: định dạng kiểu dữ liệu của data (dùng chuẩn MIME), thường được tự xác định bởi hệ thống
- **Category**: bổ sung thông tin cho các action của intent (ví dụ: nếu một activity có thuộc tính category là `CATEGORY_LAUNCHER` nghĩa là activity đó có thể khởi chạy cấp ứng dụng)
- **Extras**: dữ liệu bổ sung nếu vùng Data là chưa đủ, extras sử dụng cấu trúc bundle gồm các cặp (key, value)





UNETI



1.2 SỬ DỤNG INTENT TRAO ĐỔI DỮ LIỆU

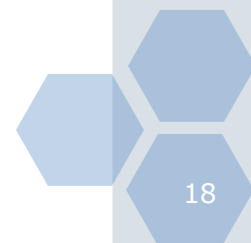




1.2 Sử dụng intent trao đổi dữ liệu

Sử dụng Intent để trao đổi dữ liệu giữa các Activity

- Dữ liệu được đóng gói vào intent:
 - Lựa chọn dữ liệu đưa vào intent phù hợp
 - Có chuẩn trao đổi dữ liệu chung để dễ xử lý
- Sau đó gọi activity phù hợp để xử lý
 - **startActivity(myIntent);**
- Nếu muốn nhận kết quả trả về thì dùng
 - **startActivityForResult(myIntent, CODE);**
 - Trong đó CODE là một số nguyên, dùng để phân biệt kết quả trả về giữa các activity khác nhau
 - Xử lý bằng cách viết lại hàm **onActivityResult**

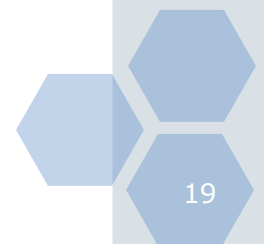




1.2 Sử dụng intent trao đổi dữ liệu

Sử dụng Intent để trao đổi dữ liệu giữa các Activity

- Về phía bên activity nhận, lấy intent gửi cho mình bằng **getIntent()**
- Nếu cần trả về kết quả nào đó cho activity trước
 - Thông báo thất bại: **setResult(RESULT_CANCELED);**
 - Thông báo thành công: **setResult(RESULT_OK, x);**
 - Với x là intent mà sẽ trả ngược lại cho activity gọi (tất nhiên là cần đưa dữ liệu vào x trước khi setResult, việc đưa dữ liệu sử dụng các hàm putExtra)
 - Chú ý trường hợp dữ liệu phức tạp: putSerializable





1.2 Sử dụng intent trao đổi dữ liệu

```
public void btnNhap(View v) {
    Intent i = new Intent(this, NhapSoActivity.class);
    startActivityForResult(i, 1001);
}

@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    if (requestCode == 1001) {
        if (resultCode == RESULT_OK) {
            TextView t1 = (TextView) findViewById(R.id.textView1);
            TextView t2 = (TextView) findViewById(R.id.textView2);
            TextView t3 = (TextView) findViewById(R.id.textView3);
            String a = data.getStringExtra("SoA");
            String b = data.getStringExtra("SoB");
            t1.setText("A = " + a);
            t2.setText("B = " + b);
            t3.setText("Tổng = " + (Integer.parseInt(a) + Integer.parseInt(b)));
            Toast.makeText(this, "Trả về thành công", Toast.LENGTH_SHORT).show();
        }
        else
            Toast.makeText(this, "Trả về thất bại", Toast.LENGTH_SHORT).show();
    }
    else
        super.onActivityResult(requestCode, resultCode, data);
}
```

Đoạn mã minh họa việc gọi activity nhập liệu và xử lý kết quả trả về

onActivityResult: được tự động gọi khi activity nhập liệu kết thúc



1.2 Sử dụng intent trao đổi dữ liệu

```
public class NhapSoActivity extends Activity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_nhap_so);  
    }  
  
    public void btnCancel(View v) {  
        setResult(RESULT_CANCELED);  
        finish();  
    }  
  
    public void btnOK(View v) {  
        Intent i = new Intent();  
        EditText t1 = (EditText) findViewById(R.id.editText1);  
        EditText t2 = (EditText) findViewById(R.id.editText2);  
        i.putExtra("SoA", t1.getText().toString());  
        i.putExtra("SoB", t2.getText().toString());  
        setResult(RESULT_OK, i);  
        finish();  
    }  
}
```

Xử lý ở phía activity nhập liệu

Dùng `setResult` để thiết lập dữ liệu trả về cho activity cha



UNETI



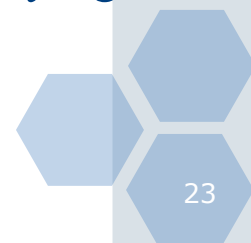
1.3 BROADCAST RECEIVERS





1.3.1 TỔNG QUAN BROADCAST RECEIVERS

- **Broadcast receiver** (gọi tắt là receiver): là một trong bốn loại thành phần cơ bản của ứng dụng android
- Receiver là một class java nhận và xử lý các sự kiện mà hệ thống (hoặc ứng dụng nào đó) phát ra
 - VD: tín hiệu báo mất wifi, tín hiệu báo cuộc gọi đến,...
- Khi hệ thống phát đi sự kiện, có 2 cơ chế phát:
 - Không thứ tự: mọi receiver đủ điều kiện đều nhận được
 - Có thứ tự: receiver nào ưu tiên hơn thì nhận trước và có thể điều chỉnh thông tin tín hiệu đến các receiver sau (Cơ chế này khá giống xử lý ngắt (interrupt) trong HĐH)





1.3.1 TỔNG QUAN BROADCAST RECEIVERS

Một số broadcast thông dụng

- Báo hệ thống khởi động xong
- Báo có package mới cài vào hoặc xóa đi
- Báo tắt máy
- Báo cắm sạc
- Báo rút sạc
- Thông báo cắm thẻ nhớ
- Thông báo rút thẻ nhớ
- Thông báo tin nhắn tới
- Thông báo có cuộc gọi đi/đến



1.3.2 VÒNG ĐỜI BROADCAST RECEIVERS

1. Khi ứng dụng được cài lên thiết bị:

- Hệ thống đọc file `AndroidManifest.xml` để xem receiver đăng ký xử lý những sự kiện nào
- Cho receiver vào danh sách các receiver cài trên thiết bị

2. Khi xảy ra sự kiện:

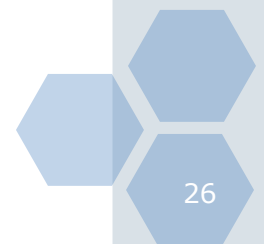
- Hệ thống tạo một intent chứa thông tin về sự kiện
- Gửi intent đó đến các receiver đăng ký xử lý sự kiện
 - Nếu sự kiện không thứ tự: gửi đồng loạt đến tất cả receiver
 - Nếu sự kiện có thứ tự: gửi lần lượt đến từng receiver
- Khi intent được gửi đến receiver: hệ thống gọi hàm `onReceive` để xử lý intent đó





1.3.2 VÒNG ĐỜI BROADCAST RECEIVERS

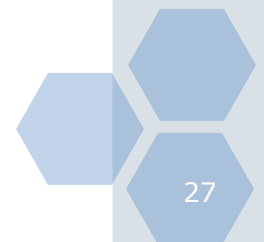
- Broadcast receiver hoạt động vô cùng đơn giản
 - Chỉ cần viết duy nhất phương thức **onReceive**
 - Khi có sự kiện mà broadcast receiver đã đăng ký nhận được phát đi, thì phương thức **onReceive** của broadcast receiver đó sẽ được gọi
 - Sau khi thực thi xong phương thức này, lifecycle của receiver kết thúc
- Trường hợp broadcast có thứ tự, ta có thể gọi hàm **abortBroadcast** trong khi **onReceive** đang chạy để ngăn không cho các receiver sau nhận broadcast





1.3.2 VÒNG ĐỜI BROADCAST RECEIVERS

- Ngay khi onReceive kết thúc, hệ thống coi như receiver đã không còn hoạt động và có thể hủy tiến trình chứa receiver này bất cứ lúc nào, vì thế:
 - Tránh xử lý các code quá lâu trong onReceive
 - Không có xử lý bất đồng bộ, chờ callback... trong receiver (cụ thể như hiển thị Dialog, kết nối service...)
 - Nếu cứ cố dùng, hệ thống sẽ thông báo ứng dụng bị waiting quá lâu, có force close hay không, đây là lỗi nên tránh vì đa số người dùng sẽ chọn YES
- Độ ưu tiên của receiver nên từ -1000 đến 1000
`<intent-filter android:priority="1000">`





1.3.3 VÍ DỤ SỬ DỤNG BROADCAST RECEIVERS

Tạo ứng dụng hiển thị % pin còn lại của thiết bị

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity"
    android:orientation="vertical">
    <TextView
        android:id="@+id/textfield"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_marginTop="40dip" />
    <ProgressBar
        android:id="@+id/progressbar"
        style="?android:attr/progressBarStyleHorizontal"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_marginTop="20dip"
        android:max="100"
        android:maxHeight="500dip"
        android:maxLength="300dip"
        android:minHeight="100dip"
        android:minWidth="200dip" />
</LinearLayout>
```



1.3.3 VÍ DỤ SỬ DỤNG BROADCAST RECEIVERS

Tạo ứng dụng hiển thị % pin còn lại của thiết bị

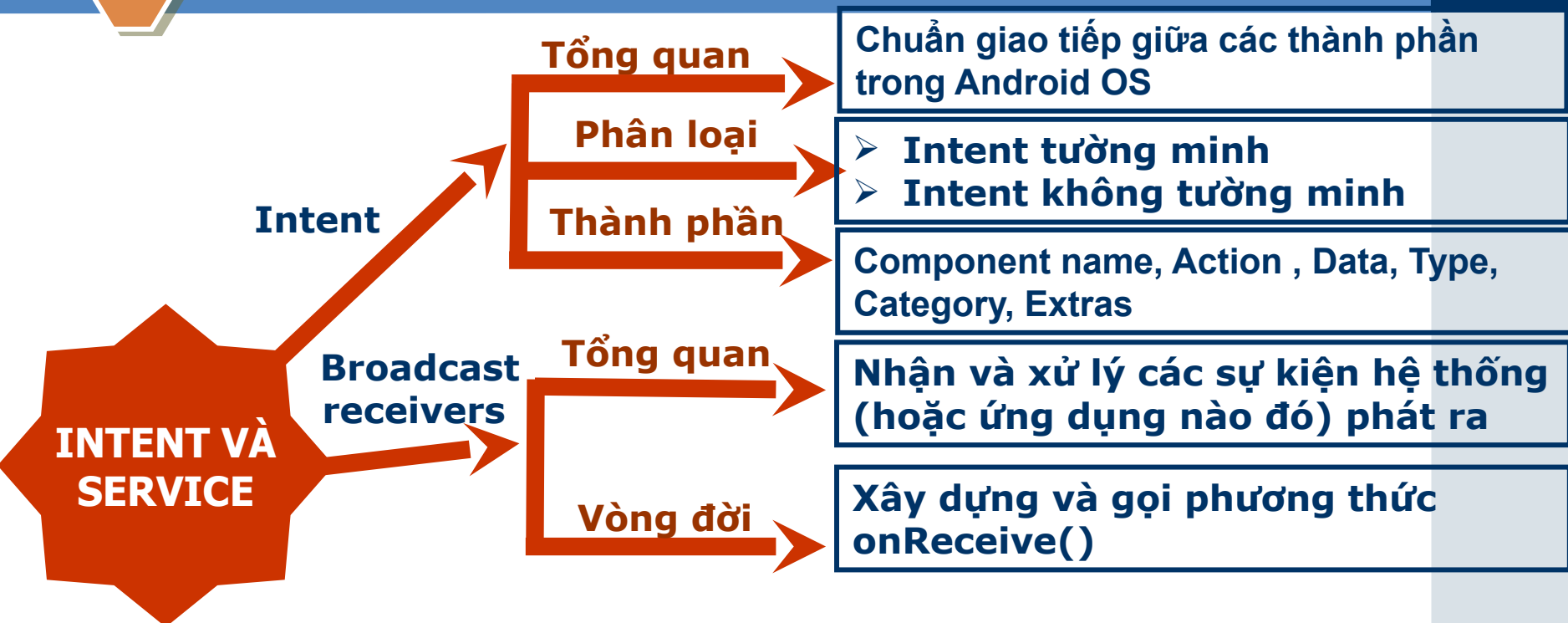
```
package com.example.bai11_vd1;
//import
public class MainActivity extends AppCompatActivity {
    private BroadcastReceiver mBatInfoReceiver = new BroadcastReceiver() {
        @Override
        public void onReceive(Context c, Intent i) {
            int level = i.getIntExtra("level", 0);
            ProgressBar pb = (ProgressBar) findViewById(R.id.progressbar);
            pb.setProgress(level);
            TextView tv = (TextView) findViewById(R.id.textfield);
            tv.setText("Battery Level: " + Integer.toString(level) + "%");
        }
    };

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        registerReceiver(mBatInfoReceiver, new
IntentFilter(Intent.ACTION_BATTERY_CHANGED));
    }
}
```





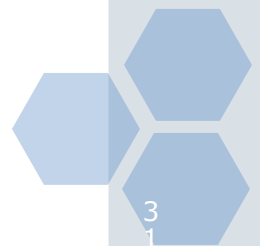
Tổng kết





Câu hỏi lý thuyết

1. Intent là gì? Nêu sự khác nhau giữa intent tường minh và intent không tường minh?.
2. Nêu các cách tạo và chuyển dữ liệu giữa các activity sử dụng intent





Bài tập

1. Xây dựng 2 Giao diện layout1, layout2. layout1 có form nhập vào 2 số a, b và 4 nút "+", "-", "x", "/". Khi click vào một nút sẽ gửi a,b qua Activity kết nối với layout2 để tính toán. layout2 có nút "Return", bấm nút "Return" thì Activity kết nối với layout2 trả lại kết quả về cho layout1 hiển thị lên TextView





Bài tập

2. Viết chương trình theo dõi % pin của điện thoại, nếu % pin của điện thoại dưới 20% sẽ hiện thị Notification thông báo cho người dùng hay bật chế độ tiết kiệm pin và sạc điện thoại.

